

Off-policy actor–critic for continuous control

DDPG $\rightarrow$ TD3 $\rightarrow$ SAC and the newer architectures — a technical reference,
mapped onto the HyMeKo RL substrate, with a safe-RL forward look

Aiko (Claude Code) — for Dr. Csaba Hajdu

2026-06-21

Abstract

A working reference for the off-policy actor–critic family, written to inform the DDPG/TD3 implementation on the HyMeKo cart-pole (PPO is the saved baseline). It traces the lineage DDPG $\rightarrow$ TD3 $\rightarrow$ SAC by the *specific failure each successor fixes*, surveys the modern sample-efficiency/stability methods (REDQ, TQC, DroQ, CrossQ, TD7), distills the design axes that actually matter, maps each onto the repo’s RolloutEnv + backbone + exploration-vocab + off-policy-core architecture, and sketches how safe RL (constrained MDPs) would enter declaratively. Synthesis from the established literature (knowledge to 2026-01); not a web-cited survey — a deep-research pass can add external citations on request.

1 The off-policy contract

On-policy methods (PPO, our baseline) collect a fresh rollout, update, and discard it. Off-policy methods keep a **replay buffer** of past transitions (s, a, r, s', d) and resample them, decoupling data collection from the update — far more sample-efficient, at the cost of stability. The shared machinery:

- **A Q -critic** $Q_\phi(s, a)$ (action-value), trained by a Bellman/TD target $y = r + \gamma(1-d)Q_{\bar{\phi}}(s', a')$ on a *target* network $\bar{\phi}$ (Polyak-averaged, $\bar{\phi} \leftarrow \tau\phi + (1-\tau)\bar{\phi}$) for a non-moving regression target.
- **A policy** π improved *through* the critic, not by a likelihood-ratio surrogate.
- **Exploration** injected explicitly (the policy is deterministic or its entropy is controlled), not implicit in the on-policy stochasticity.

The recurring enemy is **Q -value overestimation**: $\max/\arg\max$ over a noisy critic biases the target upward, the policy exploits the error, and training diverges. Most of the lineage is a sequence of overestimation controls.

2 DDPG — deterministic policy gradient

Idea. A deterministic actor $\mu_\theta(s)$ and a Q -critic; the actor ascends the critic:

$$\nabla_\theta J \approx \mathbb{E}_{s \sim \mathcal{D}} [\nabla_a Q_\phi(s, a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)].$$

Critic regresses to $y = r + \gamma(1-d)Q_{\bar{\phi}}(s', \mu_{\bar{\theta}}(s'))$. Exploration = additive noise at action selection: $a = \mu_\theta(s) + \epsilon$, $\epsilon \sim \mathcal{N}$ or Ornstein–Uhlenbeck. **Failure modes (why nobody ships plain DDPG)**: severe Q -overestimation; extreme sensitivity to hyperparameters/seed (the cart-pole’s MLP-bimodality is a mild echo); brittle target updates. It is the right *first* implementation (smallest off-policy core) but expected to be fragile — report that honestly rather than tune it away.

3 TD3 — three fixes to DDPG

TD3 = DDPG + three targeted patches, each addressing a named DDPG pathology:

1. **Clipped double-Q (twin critics).** Two critics Q_{ϕ_1}, Q_{ϕ_2} ; the target uses the *minimum*: $y = r + \gamma(1 - d) \min_i Q_{\phi_i}(s', \tilde{a}')$. Directly caps overestimation.
2. **Delayed policy updates.** Update the actor (and targets) every d critic steps ($d=2$) — a slower-moving policy on a better-fit critic.
3. **Target policy smoothing.** $\tilde{a}' = \mu_{\tilde{\theta}}(s') + \text{clip}(\epsilon, -c, c)$, $\epsilon \sim \mathcal{N}(0, \sigma)$ — regularises the critic so it can't exploit sharp peaks.

TD3 is the reliable deterministic baseline; on our codebase it is *additive over DDPG* (a second critic, a counter, one more noise site — the third injection site in the exploration vocab).

4 SAC — maximum-entropy, stochastic, off-policy

Idea. Maximise reward *plus* policy entropy: $J = \sum_t \mathbb{E}[r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))]$. A *stochastic* squashed-Gaussian actor (tanh-Gaussian) gives the reparameterised policy gradient; soft Q targets add the entropy term; twin critics (as TD3) cap overestimation; the temperature α is auto-tuned to a target entropy. **Why it matters:** SAC is the single strongest, most robust continuous-control baseline — the fair opponent to put the HSiKAN backbone into. It is also the cleanest fit for the exploration vocab: entropy is intrinsic (a source wired to the policy distribution), so PPO and SAC share that injection site while differing in the on/off-policy core.

5 Newer architectures (post-SAC: sample efficiency & stability)

The frontier is mostly *better/faster critics* at high update-to-data (UTD) ratios:

- REDQ (2021).** A *randomised ensemble* of N critics; targets take the min over a random subset of size $M=2$. Enables $\text{UTD} \approx 20$ (many gradient steps per env step) $\Rightarrow$ SAC-level returns at a fraction of the samples. Cost: N critics.
- TQC (2020).** *Truncated Quantile Critics* — distributional critics (quantile regression) whose pooled atoms are *truncated* (drop the top- k) to control overestimation continuously rather than by a hard min. Strong on harder MuJoCo.
- DroQ (2021).** REDQ's effect with a *small* ensemble by adding **dropout** + **LayerNorm** to the critics — cheaper, nearly as sample-efficient. A pragmatic default.
- CrossQ (2024).** **Removes target networks entirely**; uses BatchNorm (carefully placed) + a wider critic to stay stable. Simpler and compute-light, matching REDQ-class efficiency — attractive because it deletes the target-net machinery.
- TD7 (2023).** TD3 + **SALE** (state-action learned embeddings: a learned latent the critic/actor consume) + **LAP** (loss-adjusted prioritised replay) + policy checkpoints. State of the art among deterministic methods; the SALE embedding is conceptually close to our *backbone* idea.

Adjacent families worth knowing, not on the immediate path: **distributional RL** (C51 $\rightarrow$ QR-DQN $\rightarrow$ IQN — TQC's lineage), **model-based** (MBPO, Dreamer-v3 — learn a world model, plan/imagine), and **offline RL** (CQL, IQL — learn from a fixed buffer, no interaction).

6 The design axes that actually matter

A taxonomy to place any method (and any HyMeKo variant) on:

axis	options	who
data	on-policy / off-policy(replay)	PPO / DDPG,TD3,SAC,...
policy	deterministic / stochastic	DDPG,TD3 / SAC,PPO
critic	$V(s)$ / $Q(s,a)$ / distributional $Z(s,a)$	PPO / DDPG... / TQC,IQN
overestimation ctrl	min-of-2 / ensemble-min / quantile-trunc	TD3 / REDQ / TQC
target net	Polyak / none	most / CrossQ
exploration site	policy-entropy / action-noise / target-smoothing	PPO,SAC / DDPG,TD3 / TD3
UTD ratio	1 / high(~ 20)	classic / REDQ,DroQ
representation	raw / learned embedding (SALE) / hypergraph (HSiKAN)	most / TD7 / ours

The last row is the research seam: every method above is agnostic to *how the state is encoded*, so the HSiKAN hypergraph backbone is an orthogonal swap into any of them — which is exactly why a real-topology task (not cart-pole) is needed to test whether the structure helps.

7 Mapping onto the HyMeKo substrate

The repo already has the orthogonal pieces; off-policy adds a bounded core (architecture plan 2026-06-21-rl-algorithm-architecture):

- **Reuse:** RolloutEnv Protocol (collection), the `hsikan/mlp/signedkan` backbones (now also feeding a Q -critic that consumes `concat(embed, action)`), the vectorised env factory, `policy_store` (off-policy policies store identically).
- **New (P2):** `ReplayBuffer` (SoA ring, fixed RSS), `QCritic` + deterministic actor head, target nets + Polyak, the DPG update. **P3** (TD3) is additive: twin Q , delayed updates, target smoothing. **Later:** SAC (stochastic head + auto- α), then a REDQ/CrossQ critic option behind a flag.
- **Exploration vocab (P1):** the three injection sites (policy-dist, action-select, target-action) already cover PPO/DDPG/TD3/SAC; a method *declares* which sites it consumes. DDPG can ship with a minimal `Gaussian NoiseProcess` now and be retrofitted to the declarative vocab.

Recommended order: DDPG (smallest core, expect fragility) $\rightarrow$ TD3 (the reliable deterministic baseline) $\rightarrow$ SAC (the strong stochastic baseline) $\rightarrow$ optionally a high-UTD critic (DroQ/CrossQ) for sample efficiency. Each benchmarked against the saved PPO reference (192 ± 15 cart-pole), and the *architecture* comparison run only on a real-topology task with the params-matched control.

8 Safe reinforcement learning — the next direction

Once an off-policy learner works, “safe RL” asks it to respect constraints, not just maximise reward. The standard framings:

- **Constrained MDP (CMDP).** Maximise return subject to expected cost $\leq$ budget: $\max_{\pi} J_r(\pi)$ s.t. $J_c(\pi) \leq d$. Solved by a **Lagrangian** (a learned multiplier λ on the cost, dual ascent) — e.g. Lagrangian-SAC/TD3, or **CPO** (a trust-region constrained policy step).
- **Safety layer / shielding.** A projection or shield that overrides unsafe actions at execution (a hard constraint), optionally learned (Safety-Layer) or model-based (control-barrier functions).
- **Recovery / fallback policies.** A separate policy that activates near constraint violation.

The HyMeKo angle (why this is a natural fit): constraints are *declarative* here already. The reward is a weighted bundle of `meta_reward` terms; a **cost** is the same shape (a `meta_constraint` vocabulary: a bundle of cost terms with budgets), and the kinematic hypergraph already carries the safety-relevant structure (joint limits, self-collision, out-of-bounds — terms we *already* have as reward penalties in `galambos_task`). So a CMDP would be: read a `constraint_spec` from `.hymeko`, evaluate J_c from those terms, and add a Lagrangian multiplier to the off-policy update — the exact same “MDP-and-now-its-constraints as data” story. *Proposed as the direction after TD3; not started.*

9 Bottom line for the implementation

Build DDPG now as the minimal off-policy core (honest about its fragility), TD3 immediately after as the deterministic baseline, then SAC as the strong opponent. Keep the HSiKAN backbone as an orthogonal swap, and hold the architecture verdict for a real-topology task. Safe RL enters as a declarative `meta_constraint` + a Lagrangian on the off-policy update — the cleanest continuation of the storage thesis.

Companion artifacts. Architecture & workflow plan: `docs/plans/2026-06-21-rl-algorithm-architecture/`. PPO baseline + storage: `reports/2026-06-21-rl-on-hymeko-technical-report.{tex,pdf}`.